

1. INTEGRATED SITUATION

project description

The Republic of Rwanda is a developing nation with a decentralized administrative system composed of provinces, districts, sectors, cells, and villages. In addition to these administrative divisions, the country includes the capital city, Kigali, and six designated secondary cities: Huye, Muhanga, Musanze, Nyagatare, Rubavu, and Rusizi. The government also consists of various ministries, including the Ministry of Infrastructure.

The Ministry of Infrastructure is responsible for overseeing sustainable infrastructure development in key sectors such as transport, energy, water supply, and sanitation, housing and human settlement. Its mission is to drive Rwanda's economic growth while improving the quality of life for its citizens.

In the transport sector the ministry of infrastructure is focused on constructing and maintaining roads that connect the secondary cities to the capital city. While some roads are already in place, others are planned for future development . Below is a list of both existing roads and those scheduled for construction , along with their estimated budgets in Rwandan Francs

1. Kigali-Muhanga (28.6 Billion RWF)
2. Kigali-Musanze(28.6 Billion Rwf)
3. Kigali-Nyagatare(70.84 Billion Rwf)
4. Muhanga-Huye (56.7 Billion Rwf)
5. Musanze-Rubavu(33.7 Billion Rwf)
6. Huye-Rusizi (80.96 Billion Rwf)
7. Muhanga-Rusizi (117.5 Billion Rwf)
8. Musanze-Nyagatare(96.14 Billion Rwf)
9. Muhanga- Musanze (66.3 Billion Rwf)

Each city is assigned a numerical identifier as follows:

- > Kigali(1)
- > Huye(2)
- > Muhanga(3)
- > Musanze(4)
- > Nyagatare(5)
- > Rubavu(6)
- > Rusizi(7)

As Rwanda continues to develop , the government plans to designate additional secondary cities. Each newly added city will receive the next available index number. For example, since the last indexed city is Rusizi(7) , the next secondary city introduced will be assigned index 8 . Now challenges identified expected to be solved

1. Recording existing cities
2. Adding new cities
3. Recording existing roads(including their budgets)

4. Registering new roads
5. Modifying city details
6. Searching for a city
7. Displaying the list of cities and roads

So you as a c/c++ developer:

The Ministry of infrastructure has hired you to develop a c/c++ console application that includes the following functionalities:

1. Record the existing cities
2. Add new cities
3. Register roads between cities
4. Assign and manage road budgets
5. Modify city details
6. Search for a specific city
7. Display all recorded data on the console
8. Exit the application

this application will support the Ministry in the efficiently managing and planning Rwanda's infrastructure development

Tasks then

-->Task1:

Add new cit(ies) after specifying the number of cities to be added

--> TASK2

- a) Add roads between cities (1 if there is a road between 2 cities or 0 if there is no road between 2 cities)
- b) Add the budget for roads specified

--> TASK3

- a) Save cities in a text file cities.txt with two columns , index, and city_name . Save the roads in a text file roads.txt with two columns road and budget as recorded

--> TASK4

- a) Edit / update the city name
- b) Search for a city by entering its name

--> TASK5

- a)Use Graph to output recorded data using two adjacent matrices, one for roads and another for the budget as recorded

Instructions then :

-> In the Produce flowchart codes to be put into the mermaid so that I can get the the flowchart online

-> The program must run using the following menus:

---> MENU1: Add existing cities given in the scenario:

. Initially , this menu will be used to record 7 existing cities.

. As the program runs, the same menu will be used to add any other new secondary city(ies) follow this order

1. Add new city(ies)
2. Add roads between cities

3. Add the budget for roads
4. Edit city
5. Search for a city using its index
6. Display cities
7. Display roads
8. Display recorded data on console
9. Exit

Enter your choice : 1

Enter the number of cities to add: 7

Enter the name for city 1: Kigali

Enter the name for city 2: Huye

Enter the name for city 3: Muhanga

Enter the name for city 4: Musanze

Enter the name for city 5: Nyagatare

Enter the name for city 6: Rubavu

Enter the name for city 7: Rusizi

--> MENU2: Add roads between existing cities given in the scenario

. Use the names of two cities that you want to record the road between , as indicated below
(Below) is a menu

Menu:

1. Add new city(ies)
2. Add roads between for roads
3. Add the budget for roads
4. Edit city
5. Search for a city using its index
6. Display cities
7. Display roads
8. Display recorded data on console
9. Exit

Enter you choice: 2

Enter the name of the first city : Kigali

Enter the name of the second city : Musanze

Road added successfully between Kigali and Musanze

--> MENU3: Add road's budget

. By providing names of the cities linked by the road, the program should allow the user to enter the budget for that road. Below is the sample output
(Below is the menu then)

1. Add new city(ies)
2. Add roads between cities
3. Add the budget for roads
4. Edit city

5. Search for a city using its index
6. Display cities
7. Display roads
8. Display recorded data on console
9. Exit

Enter your choice: 3

Enter the name of the first city: Kigali

Enter the name of the second city: Musanze

Enter the budget for the road: 28.6

Budget added successfully for the road between Kigali and Musanze

--> MENU4: Edit a city

The existing name of the city is changed after providing its index as by the following sample output

(Below is the menu)

Menu:

1. Add new city(ies)
2. Add roads between cities
3. Add the budget for roads
4. Edit city
5. Search for a city using its index
6. Display cities
7. Display roads
8. Display recorded data on console
9. Exit

Enter your choice: 4

Enter the index of the city to edit: 1

Enter the new name for city: Kigari

City updated successfully

--> MENU5: Search for a city by providing its index

--> MENU6: Display cities

(Below is a response when displaying cities)

Cities:

1. Kigali
2. Huye
3. Muhanga
4. Musanze
5. Nyagatare
6. Rubavu
7. Rusizi

--> MENU7: Display roads

These should be displayed by showing cities and roads in adjacency matrix as follow:

(Below is how it should be)

Cites:

1. Kigali
2. Huye
3. Muhanga
4. Musanze
5. Nyagatare
6. Rubavu
7. Rusizi

```
0 1 0 1 1 0 0
1 0 1 0 0 0 1
0 1 0 1 0 0 1
1 0 1 0 1 1 0
1 0 0 1 0 0 0
0 0 0 1 0 0 0
0 1 1 0 0 0 0
```

--> MENU8: Display recorded data on the console:

If you enter 8 the recorded cities (from task 1) and two adjacent matrixes (one for roads and another one for a budget) will be displayed on console as indicated here:

(Below is how it should look like)

Cites:

1. Kigali
2. Huye
3. Muhanga
4. Musanze
5. Nyagatare
6. Rubavu
7. Rusizi

```
0 1 0 1 1 0 0
1 0 1 0 0 0 1
0 1 0 1 0 0 1
1 0 1 0 1 1 0
1 0 0 1 0 0 0
0 0 0 1 0 0 0
0 1 1 0 0 0 0
```

Budget adjacency matrix:

```
0.0 28.6 0.0 28.6 70.8 0.0 0.0
28.6 0.0 56.7 0.0 0.0 0.0 81.0
0.0 56.7 0.0 66.5 0.0 0.0 117.5
```

```
28.6 0.0 66.5 0.0 96.1 33.7 0.0
70.8 0.0 0.0 96.1 0.0 0.0 0.0
0.0 0.0 0.0 33.7 0.0 0.0 0.0
0.0 81.0 117.5 0.0 0.0 0.0 0.0
```

MENU9: Exit

Any time 9 is pressed , the program will stop to run

In cities.txt and roads.txt files, data are displayed as indicated below respectively:

----- Cities.txt-----

Index City_Name

- 1 Kigali
- 2 Huye
- 3 Muhanga
- 4 Musanze
- 5 Nyagatare
- 6 Rubavu
- 7 Rusizi

-----Roads.txt-----

Nbr Road Budget

1. Kigali-Muhanga 28.6
2. Kigali-Musanze 28.6
3. Kigali-Nyagatare 70.84
4. Muhanga-Huye 56.7
5. Musanze-Huye 33.7
6. Huye-Rusizi 80.96
7. Muhanga-Rusizi 117.5
8. Musanze-Nyagatare 96.14
9. Muhanga-Musanze 66.3

The following checks must be controlled

1. The city is recorded once
2. If there is no road between the 2 cities , no budget can be added

Answer :

Flowchart:[NE DSA Practical Flow Chart](#)

C++ Codes:

```
#include <iostream>
#include <vector>
#include <map>
#include <iomanip>
#include <fstream>
#include <algorithm>
```

```
using namespace std;
```

```

const int MAX_CITIES = 100;

vector<string> cities;
int roadMatrix[MAX_CITIES][MAX_CITIES] = {0};
float budgetMatrix[MAX_CITIES][MAX_CITIES] = {0.0};

int getCityIndex(const string& name) {
    for (int i = 0; i < cities.size(); i++) {
        if (cities[i] == name) return i;
    }
    return -1;
}

void addCities() {
    int n;
    cout << "Enter the number of cities to add: ";
    cin >> n;
    cin.ignore();
    for (int i = 0; i < n; i++) {
        string name;
        cout << "Enter name for city " << (cities.size() + 1) << ": ";
        getline(cin, name);
        if (getCityIndex(name) == -1) {
            cities.push_back(name);
        } else {
            cout << "City already exists. Skipping.\n";
        }
    }
}

void addRoad() {
    string c1, c2;
    cout << "Enter the name of the first city: ";
    cin >> ws; getline(cin, c1);
    cout << "Enter the name of the second city: ";
    getline(cin, c2);
    int i = getCityIndex(c1);
    int j = getCityIndex(c2);
    if (i != -1 && j != -1 && i != j) {
        roadMatrix[i][j] = roadMatrix[j][i] = 1;
        cout << "Road added between " << c1 << " and " << c2 << ".\n";
    } else {
        cout << "One or both cities are invalid.\n";
    }
}

```

```
}  
}
```

```
void addBudget() {  
    string c1, c2;  
    float budget;  
    cout << "Enter the name of the first city: ";  
    cin >> ws; getline(cin, c1);  
    cout << "Enter the name of the second city: ";  
    getline(cin, c2);  
    int i = getCityIndex(c1);  
    int j = getCityIndex(c2);  
    if (i != -1 && j != -1 && roadMatrix[i][j] == 1) {  
        cout << "Enter the budget for the road: ";  
        cin >> budget;  
        budgetMatrix[i][j] = budgetMatrix[j][i] = budget;  
        cout << "Budget added for the road between " << c1 << " and " << c2 << ".\n";  
    } else {  
        cout << "Road does not exist between the given cities.\n";  
    }  
}
```

```
void editCity() {  
    int idx;  
    cout << "Enter the index of the city to edit: ";  
    cin >> idx;  
    if (idx >= 1 && idx <= cities.size()) {  
        string newName;  
        cout << "Enter new name for city: ";  
        cin >> ws; getline(cin, newName);  
        cities[idx - 1] = newName;  
        cout << "City updated successfully.\n";  
    } else {  
        cout << "Invalid index.\n";  
    }  
}
```

```
void searchCity() {  
    int idx;  
    cout << "Enter the index of the city: ";  
    cin >> idx;  
    if (idx >= 1 && idx <= cities.size()) {  
        cout << "City " << idx << ": " << cities[idx - 1] << endl;  
    } else {
```



```

        cout << "City not found.\n";
    }
}

void displayCities() {
    for (int i = 0; i < cities.size(); i++) {
        cout << (i + 1) << ": " << cities[i] << endl;
    }
}

void displayRoads() {
    cout << "Cities:\n";
    for (int i = 0; i < cities.size(); i++) {
        cout << (i + 1) << ": " << cities[i] << endl;
    }

    cout << "Roads Adjacency Matrix:\n";
    for (int i = 0; i < cities.size(); i++) {
        for (int j = 0; j < cities.size(); j++) {
            cout << roadMatrix[i][j] << " ";
        }
        cout << endl;
    }
}

void displayAllData() {
    cout << "Cities:\n";
    displayCities();

    cout << "\nRoad adjacency matrix:\n";
    for (int i = 0; i < cities.size(); i++) {
        for (int j = 0; j < cities.size(); j++) {
            cout << roadMatrix[i][j] << " ";
        }
        cout << endl;
    }

    cout << "\nBudget adjacency matrix:\n";
    for (int i = 0; i < cities.size(); i++) {
        for (int j = 0; j < cities.size(); j++) {
            cout << fixed << setprecision(2) << budgetMatrix[i][j] << " ";
        }
        cout << endl;
    }
}

```

```

}

void saveToFile() {
    ofstream citiesFile("cities.txt");
    citiesFile << "Index City_Name\n";
    for (int i = 0; i < cities.size(); i++) {
        citiesFile << (i + 1) << " " << cities[i] << "\n";
    }
    citiesFile.close();

    ofstream roadsFile("roads.txt");
    roadsFile << "Nbr Road          Budget\n";
    int count = 1;
    for (int i = 0; i < cities.size(); i++) {
        for (int j = i + 1; j < cities.size(); j++) {
            if (roadMatrix[i][j]) {
                roadsFile << count << ". "
                    << cities[i] << "-" << cities[j]
                    << string(20 - (cities[i].length() + cities[j].length()), ' ')
                    << fixed << setprecision(2) << budgetMatrix[i][j] << "\n";
                count++;
            }
        }
    }
    roadsFile.close();
}

```

```

int main() {
    int choice;

    while (true) {
        cout << "\nMenu:\n";
        cout << "1. Add new city(ies)\n";
        cout << "2. Add roads between cities\n";
        cout << "3. Add the budget for roads\n";
        cout << "4. Edit city\n";
        cout << "5. Search for a city using its index\n";
        cout << "6. Display cities\n";
        cout << "7. Display roads\n";
        cout << "8. Display recorded data on console\n";
        cout << "9. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
    }
}

```

```
switch (choice) {  
    case 1: addCities(); break;  
    case 2: addRoad(); break;  
    case 3: addBudget(); break;  
    case 4: editCity(); break;  
    case 5: searchCity(); break;  
    case 6: displayCities(); break;  
    case 7: displayRoads(); break;  
    case 8: displayAllData(); break;  
    case 9: saveToFile(); cout << "Exiting program...\n"; return 0;  
    default: cout << "Invalid choice!\n";  
}  
}  
  
return 0;  
}
```